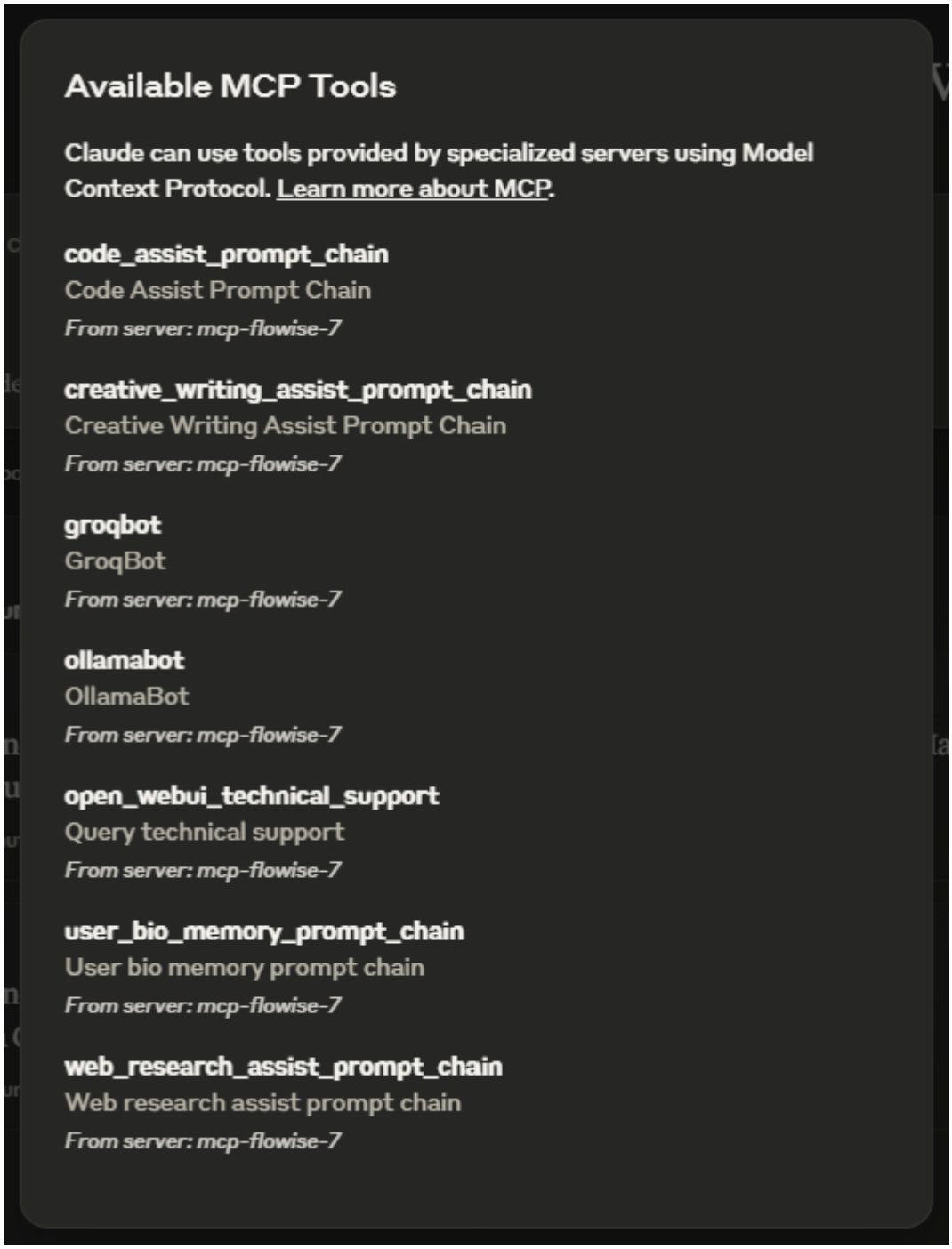
mcp-flowise

smithery.ai ▲ 16

`mcp-flowise` is a Python package implementing a Model Context Protocol (MCP) server that integrates with the Flowise API. It provides a standardized and flexible way to list chatflows, create predictions, and dynamically register tools for Flowise chatflows or assistants.

It supports two operation modes:

- **LowLevel Mode (Default)**: Dynamically registers tools for all chatflows retrieved from the Flowise API.
- **FastMCP Mode**: Provides static tools for listing chatflows and creating predictions, suitable for simpler configurations.



Features

- **Dynamic Tool Exposure**: LowLevel mode dynamically creates tools for each chatflow or assistant.
- **Simpler Configuration**: FastMCP mode exposes `list_chatflows` and `create_prediction` tools for minimal setup.
- **Flexible Filtering**: Both modes support filtering chatflows via whitelists and blacklists by IDs or names (regex).
- **MCP Integration**: Integrates seamlessly into MCP workflows.

Installation

Installing via Smithery

To install mcp-flowise for Claude Desktop automatically via Smithery:

```
npx -y @smithery/cli install @matthewhand/mcp-flowise --client claude
```

Prerequisites

- Python 3.12 or higher
- `uvx` package manager

Install and Run via `uvx`

Confirm you can run the server directly from the GitHub repository using `uvx`:

```
uvx --from git+https://github.com/andydukes/mcp-flowise mcp-flowise
```

Adding to MCP Ecosystem (`mcpServers` Configuration)

You can integrate `mcp-flowise` into your MCP ecosystem by adding it to the `mcpServers` configuration. Example:

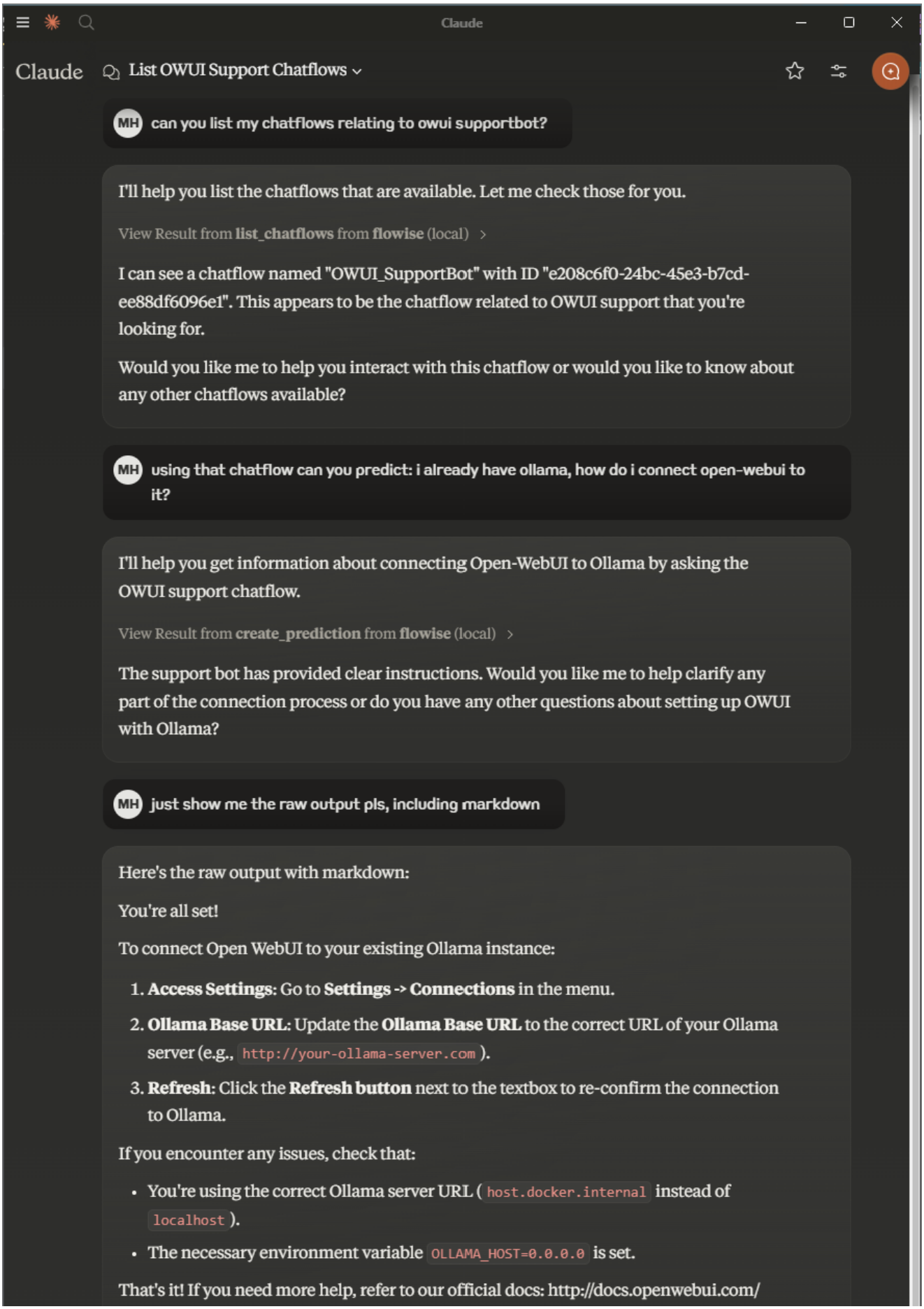
```
{
    "mcpServers": {
        "mcp-flowise": {
            "command": "uvx",
            "args": [
                "--from",
                "git+https://github.com/andydukes/mcp-flowise",
                "mcp-flowise"
            ],
            "env": {
                "FLOWISE_API_KEY": "${FLOWISE_API_KEY}",
                "FLOWISE_API_ENDPOINT": "${FLOWISE_API_ENDPOINT}"
            }
        }
    }
}
```

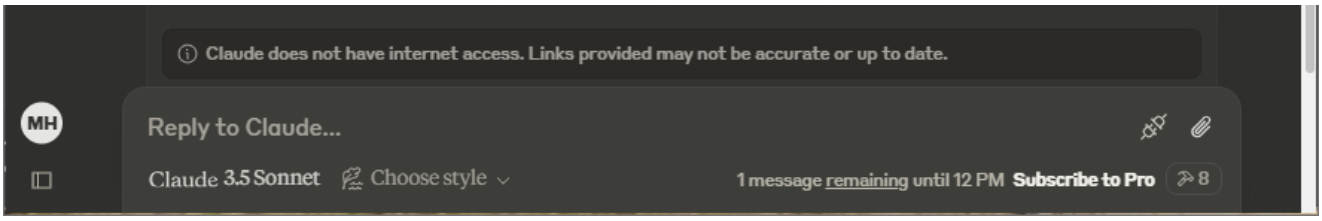
Modes of Operation

1. FastMCP Mode (Simple Mode)

Enabled by setting `FLOWISE_SIMPLE_MODE=true`. This mode:

- Exposes two tools: `list_chatflows` and `create_prediction`.
- Allows static configuration using `FLOWISE_CHATFLOW_ID` or `FLOWISE_ASSISTANT_ID`.
- Lists all available chatflows via `list_chatflows`.





2. LowLevel Mode (FLOWISE_SIMPLE_MODE=False)

Features:

- Dynamically registers all chatflows as separate tools.
- Tools are named after chatflow names (normalized).
- Uses descriptions from the `FLOWISE_CHATFLOW_DESCRIPTIONS` variable, falling back to chatflow names if no description is provided.

Example:

- `my_tool(question: str) -> str` dynamically created for a chatflow.

Running on Windows with `uvx`

If you're using `uvx` on Windows and encounter issues with `--from git+https`, the recommended solution is to clone the repository locally and configure the `mcpServers` with the full path to `uvx.exe` and the cloned repository. Additionally, include `APPDATA`, `LOGLEVEL`, and other environment variables as required.

Example Configuration for MCP Ecosystem (`mcpServers` on Windows)

```
{
  "mcpServers": {
    "flowise": {
      "command": "C:\\Users\\matth\\.local\\bin\\uvx.exe",
      "args": [
        "--from",
        "C:\\Users\\matth\\downloads\\mcp-flowise",
        "mcp-flowise"
      ],
      "env": {
        "LOGLEVEL": "ERROR",
        "APPDATA": "C:\\Users\\matth\\AppData\\Roaming",
        "FLOWISE_API_KEY": "your-api-key-goes-here",
        "FLOWISE_API_ENDPOINT": "http://localhost:3002/"
      }
    }
  }
}
```

Notes

- **Full Paths**: Use full paths for both `uvx.exe` and the cloned repository.
- **Environment Variables**: Point `APPDATA` to your Windows user profile (e.g., `C:\\Users\\<username>\\AppData\\Roaming`) if needed.
- **Log Level**: Adjust `LOGLEVEL` as needed (`ERROR`, `INFO`, `DEBUG`, etc.).

Environment Variables

General

- `FLOWISE_API_KEY`: Your Flowise API Bearer token (**required**).
- `FLOWISE_API_ENDPOINT`: Base URL for Flowise (default: `http://localhost:3002`).

LowLevel Mode (Default)

- `FLOWISE_CHATFLOW_DESCRIPTIONS`: Comma-separated list of `chatflow_id:description` pairs. Example:

  ```
  FLOWISE_CHATFLOW_DESCRIPTIONS="abc123:Chatflow One,xyz789:Chatflow Two"
  ```

FastMCP Mode (`FLOWISE_SIMPLE_MODE=true`)

- `FLOWISE_CHATFLOW_ID`: Single Chatflow ID (optional).
- `FLOWISE_ASSISTANT_ID`: Single Assistant ID (optional).
- `FLOWISE_CHATFLOW_DESCRIPTION`: Optional description for the single tool exposed.

Filtering Chatflows

Filters can be applied in both modes using the following environment variables:

- **Whitelist by ID**: `FLOWISE_WHITELIST_ID="id1,id2,id3"`
- **Blacklist by ID**: `FLOWISE_BLACKLIST_ID="id4,id5"`
- **Whitelist by Name (Regex)**: `FLOWISE_WHITELIST_NAME_REGEX=".*important.*"`
- **Blacklist by Name (Regex)**: `FLOWISE_BLACKLIST_NAME_REGEX=".*deprecated.*"`

> **Note**: Whitelists take precedence over blacklists. If both are set, the most restrictive rule is applied.

-

Security

- **Protect Your API Key**: Ensure the `FLOWISE_API_KEY` is kept secure and not exposed in logs or repositories.
- **Environment Configuration**: Use `.env` files or environment variables for sensitive configurations.

Add `.env` to your `.gitignore`:

```
# .gitignore
.env
```

Troubleshooting

- **Missing API Key**: Ensure `FLOWISE_API_KEY` is set correctly.
- **Invalid Configuration**: If both `FLOWISE_CHATFLOW_ID` and `FLOWISE_ASSISTANT_ID` are set, the server will refuse to start.
- **Connection Errors**: Verify `FLOWISE_API_ENDPOINT` is reachable.

License

This project is licensed under the MIT License. See the LICENSE file for details.

TODO

- [x] Fastmcp mode
- [x] Lowlevel mode
- [x] Filtering
- [x] Claude desktop integration
- [ ] Assistants